

농산물 중금속 부적합 예측 — 작성 보고서

0 기본 정보

이름 원병선 조 1조 제출일 2026-07-21 GitHub REPO <https://github.com/Fullstack-Tests/DBMS-Setup>

과정명	(디지털 컨버전스) 공공데이터 융합 풀스택 개발자 양성과정E
능력단위	공공데이터 융합 DBMS 구축 / 7수준
평가방법	문제해결시나리오 (100점)
평가일자	2026-07-21 (09:00 ~ 18:00)

제출방법

PPT 제출 (실행화면 캡처 포함) · 소스 폴더(01 / 02 / 03) GITHUB REPO 링크 제출

1 공공데이터 검색·확보

1-1 어떤 데이터를 골랐는가

데이터셋명 농산물 중금속 분석결과

공공데이터포털 URL <https://www.data.go.kr/data/15047544/fileData.do>

검색에 쓴 키워드 중금속 확장자 필터 CSV

1-2 왜 이 데이터를 골랐는가 (선정 이유)

유통 단계에서 중금속 부적합을 사전에 예측할 수 있으면 폐기·회수 비용을 줄이고 소비자 안전에도 도움이 될 거라 판단했다. 공공데이터포털에서 '중금속' 키워드로 검색했을 때 68,106건 규모의 실측 검사 데이터를 CSV로 제공해, 실제 모델 학습에 쓸 만한 신뢰도와 분량을 갖췄다고 봐 최종 선정했다.

1-3 데이터 선정 조건 3가지를 만족하는지 확인

행 단위 데이터인가(1행 = 검사 1건) 예

타겟이 두 가지 값(이진분류)이 되는가 예

통계·집계표가 아닌가 예

2 구조·유형·척도 파악

2-1 구조 — 실행 결과를 보고 채운다

```
df = pd.read_csv('data/농산물_중금속_분석결과.csv', encoding='cp949')  
print('데이터 크기(행, 열):', df.shape)
```

행 68,106 개 열 9 개

2-2 주요 열의 유형·척도

열 이름	유형(문자/숫자/날짜)	척도(명목/순서/등간/비율)	입력으로 쓸까
품목명	문자	명목	사용(상위 30 + 기타)
수거단계	문자	명목	사용
생산자 주소	문자	명목	시도만 추출해 사용
재배면적	숫자	비율	사용(결측 채움)
분석결과	문자	명목	제외(타겟 재료)

2-3 분석 가능하다고 판단한 근거

품목명·수거단계·생산자 주소 등 명목형 열은 인코딩을 거치면 모델 입력으로 쓸 수 있고, 재배면적처럼 비율척도인 수치형 열은 결측만 처리하면 바로 사용 가능해 전처리 후 분석이 가능하다고 판단했다. 타겟이 될 '분석결과' 열도 명목형이지만 값이 적합/부적합으로 뚜렷이 구분돼 이진분류 타겟으로 변환하기 용이했다.

3 출처·제공기관·갱신주기·이용허락범위

찾는 곳

공공데이터포털 데이터 상세페이지의 **파일데이터 정보** 표에 제공기관·갱신주기·이용허락범위가 모두 적혀 있다.

3-1 상세페이지에서 그대로 옮겨 적는다

출처	공공데이터포털(data.go.kr)-농림축산식품부 국립농산물품질관리원_농산물 중금속분석결과
제공기관	농림축산식품부 국립농산물품질관리원
갱신주기	반기
이용허락범위	이용허락범위 제한 없음
전체 건수	68,106
등록일 / 수정일	2026-04-23 / 2026-06-18

4 품질 진단

배점이 가장 큰 항목

네 가지를 모두 수치로 제시해야 한다. "없었다" 도 확인해 봤다는 근거(코드·출력)가 있어야 인정된다.

4-1 결측치 — 몇 개인지 세어서 적는다

```
print(df.isnull().sum())
```

결측이 있는 열과 개수(비율) 재배면적 19,932개(29.3%), 조사물량 12,975개(19.0%)

처리 방향 제거하면 3할이 날아가므로 중앙값으로 채운다. 단 분할한 뒤 학습 데이터의 중앙값으로 채운다

4-2 이상치 — 수치형 열의 분포를 보고 판단한다

```
print(df[['재배면적','조사물량']].describe()) # 최소·최대·사분위수 확인
```

이상해 보이는 값이 있는 열과 근거 재배면적 최대 1,000,015,010㎡, 조사물량 최대 15,002,016kg — 물리적으로 불가능해 입력 오류로 추정됨(10,000㎡ 초과 492건)

처리 방향(제거 / 유지 / 상한 자르기)과 이유 유지 — 트리 모델이라 극단값에 크게 흔들리지 않는다

4-3 표기 불일치 — 같은 것이 두 이름으로 적혀 있지 않은가

```
print(df['시도'].value_counts()) # 값을 전부 찍어 눈으로 확인
```

불일치 사례 '충남'(5,933)과 '충청남도'(5,316)가 따로 집계됨. 경북/경상북도 등 16개 시도가 같은 문제
통일 후 범주 수 33 종 → 17 종

4-4 희소 범주 — 몇 건 안 되는 범주가 많지 않은가

```
for c in ['품목명','수거단계','재배양식','시도']:
    print(c, df[c].nunique(), '종')
```

범주가 많은 열과 종 수 품목명 518종, 수거단계 4종, 재배양식 10종, 시도 17종

묶는 기준(상위 N개 + 기타) 품목은 상위 30개 + 기타(기타 비중 24.0%), 재배양식은 상위 5개 + 기타

4-5 정제 방향 정리

먼저 표기 불일치를 통일해 같은 지역이 서로 다른 이름으로 집계되지 않게 정리한다. 다음으로 희소 범주를 상위 N개 + 기타로 묶어 인코딩 시 차원이 과도하게 늘어나는 것을 막는다. 이상치는 물리적으로 불가능한 수준의 값으로 입력 오류로 추정되나, 트리 모델 특성상 스펙향이 제한적이라 제거하지 않고 그대로 둔다. 마지막으로 결측치는 학습·검증 분할을 먼저 수행한 뒤 학습 데이터의 중앙값으로 채워, 검증 데이터의 정보가 학습에 새어 들지 않도록 한다

5 타겟 정의·독립변수 선별

5-1 타겟(종속변수) 정의

```
TARGET = '부적합'
LABELS = ['적합', '부적합']
df[TARGET] = (~df['분석결과'].astype(str).str.startswith('적합')).astype(int)
```

정의 기준을 말로 쓰면 분석결과가 '적합'으로 시작하지 않으면 부적합(1). '부적합 (폐기)' 749건 + '부적합(회수폐기...)' 12건

5-2 클래스 분포 — 균형인가 불균형인가

0(적합) 67,345 건 1(부적합) 761 건 양성 비율 1.12 %

이 분포가 뒤(분할·학습·지표)에 미치는 영향 심한 불균형이라 정확도는 못 믿는다. stratify 로 분할하고 class_weight 를 준다

5-3 데이터 누수가 되는 컬럼을 제외한 근거

누수 주의

타겟을 구성하는 컬럼과 사건 종료 후 확정되는 사후 결과값은 **반드시** 입력에서 뺀다. ROC-AUC 가 0.99 처럼 비정상적으로 높으면 누수를 의심한다.

뺀 컬럼 분석결과

뺀 이유 타겟을 만든 열 자체다. 넣으면 정답을 그대로 알려주는 셈이 된다

5-4 남긴 독립변수와 선정 근거

최종 입력 7 개 : item, stage, method, sido, month, area, volume

item, stage, method, sido, month, area, volume 7개는 모두 시료 채취 시점에 이미 확정되는 값이라 예측 시점에 입력 가능하고, 분석결과 같은 사후 값이 아니라 5-3의 누수 기준에도 맞는다. 품목·재배양식·지역별 토양·환경 차이, 수거단계별 오염 노출 정도, 계절(월)에 따른 조건 변화가 부적합 여부와 관련 있다고 판단해 남겼다.

6 정제·가공하여 분석 가능한 형태로 구축

6-1 표기 통일

```
SIDO = {'충남': '충청남도', '경북': '경상북도', ...}  
df[' 시도 '] = df[' 시도 '].replace(SIDO)
```

결과 33 종 → 17 종

6-2 희소 범주 묶기

```
top = df[' 품목명 '].value_counts().head(30).index  
df[' 품목 '] = np.where(df[' 품목명 '].isin(top), df[' 품목명 '], '기타')
```

6-3 인코딩 — 글자를 숫자로

중요

코드 열 이름은 **영문**으로 만든다. 이 이름이 그대로 API 의 JSON 키가 되는데 한글 키는 터미널·도구에
서 인코딩이 깨져 오류가 난다. 보이는 이름은 `label` 이 담당한다.

인코딩 순서는 `feature_spec.json` 의 `options` 배열과 **반드시 일치**해야 한다. 화면은 선택지의 **인
덱스**를 보내기 때문이다.

```
OPTIONS[c] = sorted(df[c].astype(str).unique())  
df[CODE[c]] = df[c].astype(str).map({v: i for i, v in enumerate(OPTIONS[c])})
```

정렬 함수 · 인덱스 부여 함수를 채운 이유 options 순서와 코드가 어긋나면 화면에서 고른 것과 다른 값으로
예측된다

6-4 8:2 분할 — 클래스 분포를 고려한다

```
X_tr, X_te, y_tr, y_te = train_test_split(  
    X, y, test_size=0.2, random_state=42, stratify=y)
```

학습 54,484 행 / 검증 13,622 행

stratify 를 쓴 이유 양성이 1.12% 뿐이라 안 쓰면 검증에 부적합이 거의 안 들어갈 수 있다

6-5 결측치 채우기는 언제 하는가

누수 주의

중앙값·평균으로 채울 때 **전체 데이터의 중앙값**을 쓰면 검증 데이터의 정보가 학습에 새어 든다. 반드시 **분할한 뒤 학습 데이터의 값**으로 채운다.

채우기를 분할 후 에 한 이유 전체 중앙값을 쓰면 검증 데이터의 정보가 학습에 새어 든다. 학습 데이터의 중앙값(면적 1432, 물량 500)으로 채웠다

6-6 학습

```
model = RandomForestClassifier(n_estimators=100, max_depth=8,
                               random_state=42, class_weight='balanced')

model.fit(X_tr, y_tr)
```

class_weight 를 쓴 / 쓰지 않은 이유 양성이 1.12% 뿐이라 안 주면 전부 '적합'으로만 찍어 부적합을 하나도 못 잡는다

6-7 성능지표 4종과 지표 선택의 근거

```
acc = accuracy_score(y_te, pred)
auc = roc_auc_score(y_te, proba)
print(classification_report(y_te, pred, target_names=LABELS)) # Precision·Recall
```

지표	값	이 값을 어떻게 읽는가
정확도	0.6519	기준선 0.9888 보다 낮다 — 믿을 수 없다
ROC-AUC	0.8147	0.5(찍기)보다 훨씬 높다 — 실력이 있다
Precision(양성)	0.03	부적합이라 한 것 중 3%만 맞다 — 헛짚음이 많다
Recall(양성)	0.86	실제 부적합의 86%를 잡아냈다

전부 다수 클래스로 찍었을 때의 정확도(기준선) 0.9888

양성이 1.12%뿐이라 전부 '적합'으로 찍어도 정확도가 98.88%가 나오므로, 정확도만으로는 모델이 실제로 부적합을 잡아내는지 알 수 없다. 그래서 클래스 비율에 영향받지 않는 ROC-AUC(0.8147)를 주지표로 삼았다. 이 서비스에서는 실제 부적합을 적합으로 놓치는 것(false negative)이 가장 치명적 이므로 Precision보다 Recall(0.86)을 더 중요하게 본다. Precision이 0.03으로 낮아 오탐이 많더라도, 위험 신호를 놓치지 않고 재검사로 넘기는 쪽이 안전하기 때문이다.

7 모델·입력정의서 저장

7-1 저장 코드

```
joblib.dump(model, '../02/ model.pkl ')

spec = {'title': TITLE, 'target_labels': LABELS,
        'accuracy': round(acc, 4), 'roc_auc': round(auc, 4),
        'features': FEATURES}
json.dump(spec, open('../02/ feature_spec.json ', 'w', encoding='utf-8'),
          ensure_ascii=False)
```

7-2 feature_spec.json 에 들어가는 4가지 키

각 입력마다 name · label · kind · options

이 파일이 왜 필요한가 03 의 입력 화면이 이 파일을 읽어 폼을 자동으로 만든다

7-3 버전 일치 — 저장 환경과 서비스 환경

구축 요건

requirements.txt 의 버전은 모델을 학습한 환경과 **일치**해야 한다. 다르면 02 서버가

`ModuleNotFoundError` 로 뜨지 않는다.

라이브러리	학습 환경(01 출력)	requirements.txt	일치
python	3.11.0	3.11-slim	일치
numpy	2.4.6	2.4.6	일치
pandas	3.0.3	3.0.3	일치
scikit-learn	1.9.0	1.9.0	일치
joblib	1.5.3	1.5.3	일치

8 모델 API 서비스

8-1 엔드포인트 3개

메서드	경로	하는 일	응답에 무엇이 담기나
GET	/health	서버가 살아있는지	status UP, 모델 제목
GET	/spec	입력 정의 제공	title, target_labels, features
POST	/predict	예측(저장 안 함)	prediction, label, proba

8-2 예측 코드

```
df = pd.DataFrame([input_data])
return model.predict(df)[0], model.predict_proba(df)[0]
```

두 번째 함수가 돌려주는 것 클래스별 확률 배열 [0일 확률, 1일 확률]

요청 본문에 담은 값은 선택지의 인덱스 다 (문자열이 아니다). 그 이유 모델이 인코딩된 숫자로 학습했다. 문자열을 보내면 500 이 난다

8-3 오류 처리

없는 이력을 조회하면 404 Not Found 를 돌려줘야 한다. 그 코드 404

```
if row is None:
    raise HTTPException(status_code=404, detail='해당 예측 이력이 없습니다')
```

직접 호출해 확인한 결과 GET /predictions/999999 -> 404 해당 예측 이력이 없습니다

8-4 포트와 Swagger

이 모델 API 의 포트 8002 Swagger 주소 http://localhost:8002/docs

Swagger 에 보이는 엔드포인트 총 8 개

9 예측 이력 CRUD

9-1 CRUD 엔드포인트 5개

기능	메서드	경로	확인한 결과
C 생성	POST	/predictions	201, id=1
R 목록	GET	/predictions	200, 목록 2건
R 단건	GET	/predictions/{pid}	200, 단건
U 수정	PUT	/predictions/{pid}	200, 재예측됨
D 삭제	DELETE	/predictions/{pid}	200, deleted 1

9-2 수정 시 재예측 — 구축 요건

구축 요건

이력 수정 시 값만 바꾸는 것이 아니라 **새 입력으로 다시 예측**되어 결과가 갱신되어야 한다.

```
@app.put('/predictions/{pid}')
def update_prediction(pid: int, req: PredictRequest):
    result = run_model(req.features)    # 여기서 다시 예측한다
    ...
```

수정 전 라벨 적합 (확률 0.9998) → 수정 후 라벨 부적합 (확률 0.5898)

이 함수를 호출하지 않으면 무엇이 잘못되나 입력만 바뀌고 결과는 옛 예측이 남아 이력이 서로 안 맞게 된다

9-3 저장 확인

이력이 저장되는 곳 02/store.py 의 sqlite DB (predictions 테이블)

삭제 후 다시 조회하면 404

10 컨테이너 통합·인증

10-1 docker compose 로 뜨는 컨테이너 5개

서비스	컨테이너명	포트	하는 일
DB	db-container	3330:3306	MySQL, 회원 정보
Redis	redis-container	6380:6379	캐시
ML	ml-container	8002:8002	FastAPI, model.pkl 예측
BN	bn-container	8080:8080	Spring Boot, 인증 + ML 중계
FN	fn-container	3000:80	React 화면

기동 명령 `docker compose up -d --build`

10-2 BN 이 ML 을 부르는 주소

```
@Value("${ml.url:http://ml-container:8002}")
private String mlUrl;
```

localhost 가 아니라 컨테이너 이름을 쓰는 이유 컨테이너 안에서 localhost 는 자기 자신이다. 같은 네트워크의 서비스명으로 불러야 한다

10-3 인증 — 구축 요건

구축 요건

`/api/**` 경로는 **로그인해야 호출**되어야 한다(템플릿의 인증 설정 유지).

로그인 전에 `/api/spec` 을 부르면 상태코드 **401**

로그인 전에 예측 화면에 들어가면 어떻게 되나 예측 폼이 뜨지 않고 로그인 페이지로 이동한다

SecurityConfig 에서 이걸 강제하는 한 줄 `auth.anyRequest().authenticated()`

10-4 입력 화면 자동 생성 — 구축 요건

구축 요건

입력 화면은 `feature_spec.json` 을 읽어 자동 생성되어야 한다(하드코딩 불가).

```
{spec.features.map((f) => (  
  f.kind === 'num' ? <input type="number" .../>  
    : <select>{f.options.map((o, i) => <option value={i}>{o}  
</option>)}</select>  
)})}
```

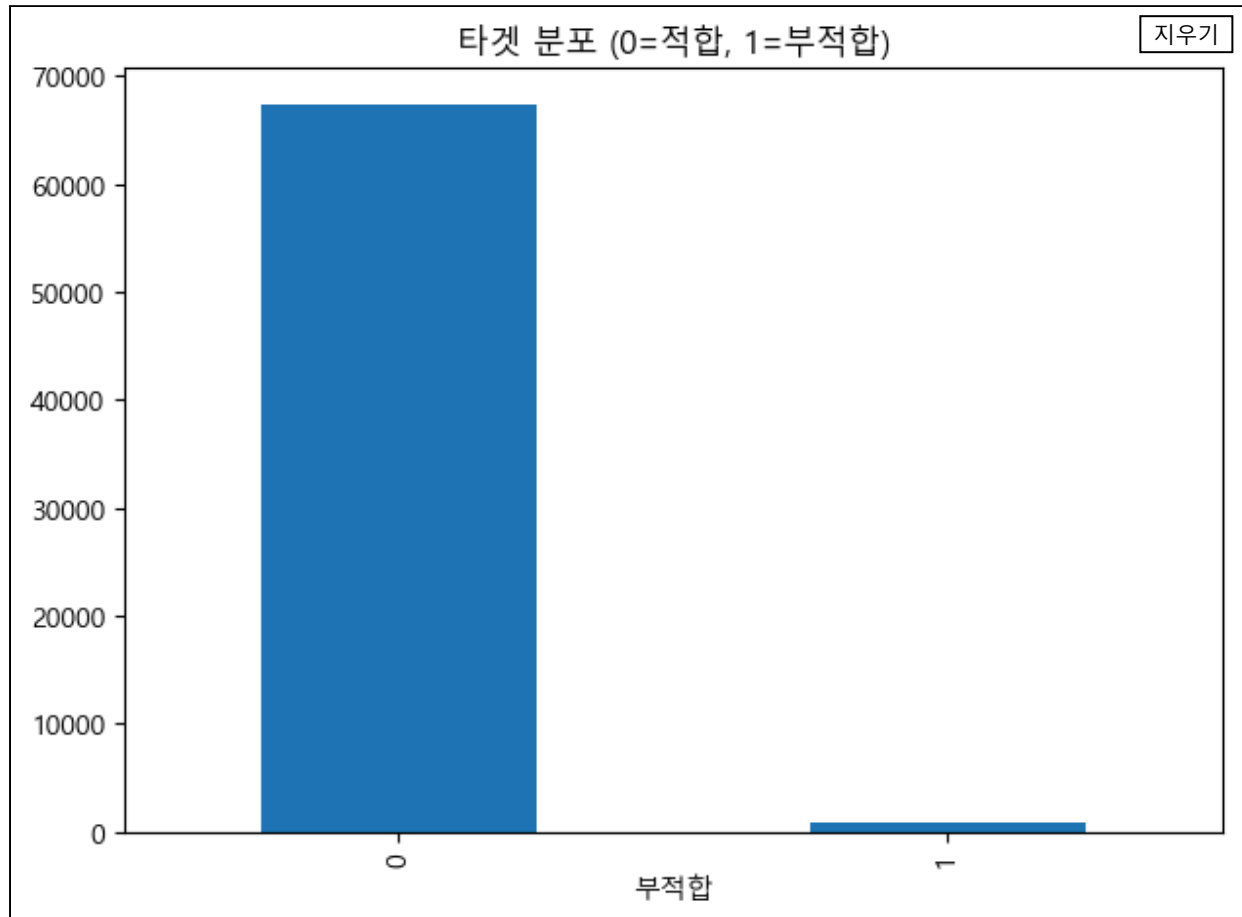
이 화면에 자동으로 생긴 입력 칸 7 개 (select 4 + number 3)

11-1 실행화면 캡처 (PPT 에 넣을 것)

넣는 법

캡처한 이미지 파일을 아래 상자에 **끌어다 놓는다**(상자를 눌러 파일을 골라도 된다). 넣은 이미지는 이 브라우저에 저장되고 인쇄·PDF 에도 함께 나온다.

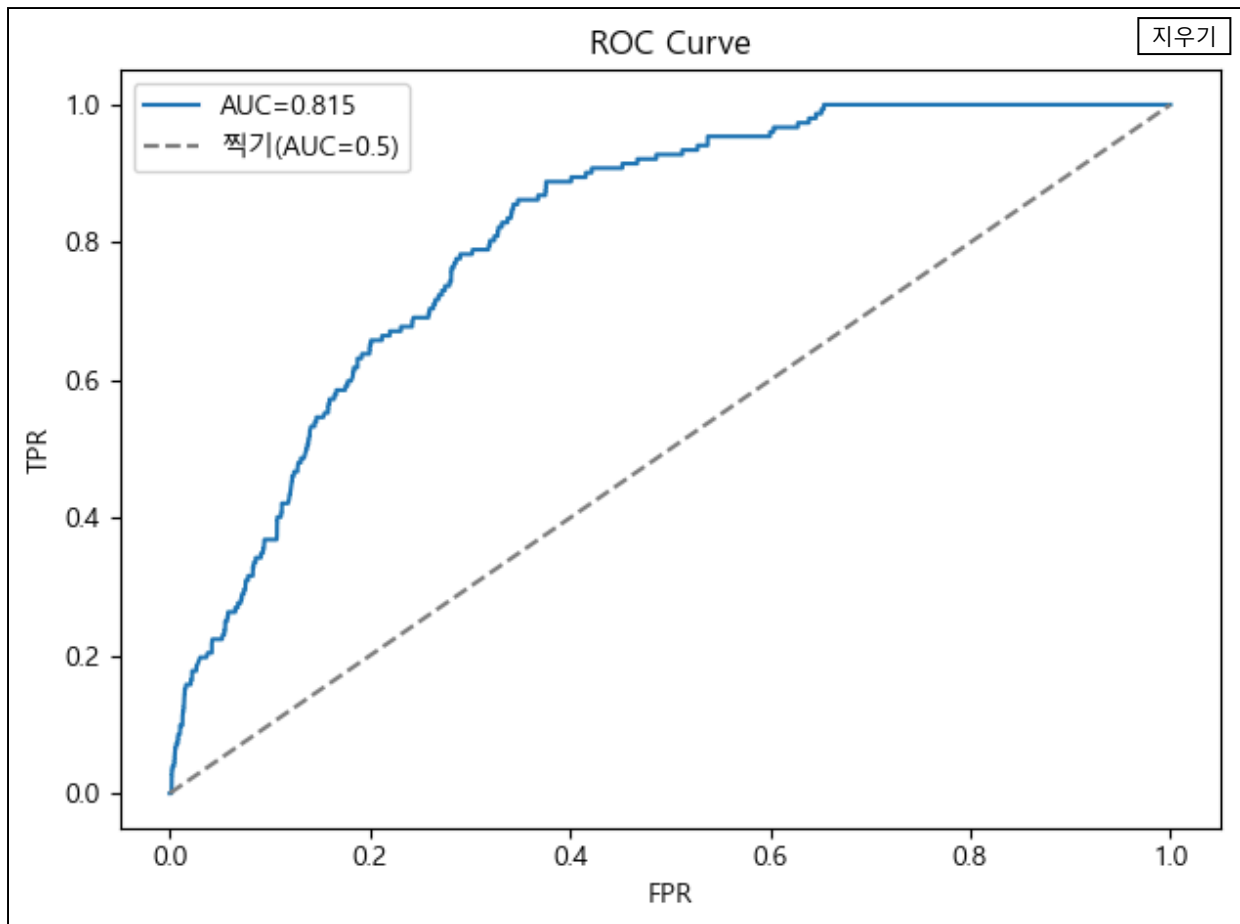
1. 품질 진단 타겟 분포



파일명 01-1_타겟분포.png

확인한 것 적합 67,345 vs 부적합 761 — 양성 1.12% 불균형

2. ROC 곡선



파일명 01-2_ROC곡선.png

확인한 것 ROC-AUC 0.8147

3. Swagger

지우기

농산물 중금속 부적합 예측 ML API

0.1.0 OAS 3.1

/openapi.json

model.pkl 을 서빙하는 예측 엔드포인트 + 예측 이력 CRUD. Spring Boot(BN)가 호출한다.

default

GET /health Health

GET /spec Get Spec

POST /predict Do Predict

POST /predictions Create Prediction

GET /predictions List Predictions

GET /predictions/{pid} Get Prediction

PUT /predictions/{pid} Update Prediction

DELETE /predictions/{pid} Delete Prediction

Schemas

HTTPValidationError > Expand all object

PredictRequest > Expand all object

ValidationError > Expand all object

파일명 02-1_Swagger.png

확인한 것 엔드포인트 8개

4. 로그인

[MAIN](#) | [USER](#) | [PREDICT](#) | [JOIN](#) | [LOGIN](#) | [LOGOUT](#) |

지우기

LOGIN PAGE

USERNAME :

PASSWORD :

파일명 03-3_로그인.png

확인한 것 로그인해야 예측 가능

5. 예측 품(자동생성)

[MAIN](#) | [USER](#) | [PREDICT](#) | [JOIN](#) | [LOGIN](#) | [LOGOUT](#) |

지우기

농산물 중금속 부적합 예측

정확도 65.2% · ROC-AUC 0.8147

새 예측

품목 :

가을무(김장무)

수거단계 :

생산

재배양식 :

GAP(인증)

생산지(시도) :

강원도

검사 월 :

9

재배면적(m^2) :

1432

조사물량(kg) :

500

예측하고 저장

예측 이력 (0)

ID	결과	입력	시각	관리
이력이 없습니다. 위에서 예측을 해보세요.				

파일명 03-4_예측품_자동생성.png

확인한 것 spec 을 읽어 품이 자동 생성됨

6. 예측 결과

[MAIN](#) [USER](#) [PREDICT](#) [JOIN](#) [LOGIN](#) [LOGOUT](#)

지우기

농산물 중금속 부적합 예측

정확도 65.2% · ROC-AUC 0.8147

새 예측

품목 : 가을무(김장무)

수거단계 : 생산

재배양식 : GAP(인증)

생산지(시도) : 강원도

검사 월 : 9

재배면적(n²) : 1432

조사물량(kg) : 500

예측하고 저장

예측 결과 : 부적합

{

"적합" : 0.4102,

"부적합" : 0.5898

}

예측 이력 (1)

ID	결과	입력	시각	관리
6	부적합	{"item":0,"stage":1,"method":1,"sido":1,"month":11,"area":10000,"volume":100}	2026-07-15T04:53:51	수정삭제

파일명 03-5_고위험_예측결과.png

확인한 것 가을무·유통/판매·경기도 -> 부적합 0.5898

7. 이력 수정 후

[MAIN](#) [USER](#) [PREDICT](#) [JOIN](#) [LOGIN](#) [LOGOUT](#) |

지우기

농산물 중금속 부적합 예측

정확도 65.2% · ROC-AUC 0.8147

새 예측

품목 :

가을무(김장무)

수거단계 :

생산

재배양식 :

GAP(인증)

생산지(시도) :

강원도

검사 월 :

9

재배면적(m^2) :

1432

조사물량(kg) :

500

예측하고 저장

예측 결과 : 부적합

```
{
  "적합": 0.4102,
  "부적합": 0.5898
}
```

예측 이력 (2)

ID	결과	입력	시각	관리
7	부적합	{"item":0,"stage":1,"method":1,"sido":1,"month":11,"area":10000,"volume":100}	2026-07-15T04:53:53	수정삭제
6	부적합	{"item":0,"stage":1,"method":1,"sido":1,"month":11,"area":10000,"volume":100}	2026-07-15T04:53:51	수정삭제

파일명 03-8_수정후.png

확인한 것 수정 시 새 입력으로 재예측됨

8. 삭제 후

[MAIN](#) [USER](#) [PREDICT](#) [JOIN](#) [LOGIN](#) [LOGOUT](#)

지우기

농산물 중금속 부적합 예측

정확도 65.2% · ROC-AUC 0.8147

새 예측

품목 :

수거단계 :

재배양식 :

생산지(시도) :

검사 월 :

재배면적(m^2) :

조사물량(kg) :

예측 결과 : 부적합

```
{
  "적합": 0.4102,
  "부적합": 0.5898
}
```

예측 이력 (1)

ID	결과	입력	시각	관리
6	부적합	{"item":0,"stage":1,"method":1,"sido":1,"month":11,"area":10000,"volume":100}	2026-07-15T04:53:51	수정 삭제

파일명 03-9_삭제후.png

확인한 것 삭제 후 이력이 줄어듦

11-2 예측 결과를 데이터 근거로 해석

이게 핵심이다

"모델이 그렇게 나왔다" 는 해석이 아니다. 품질 진단 단계에서 본 수치와 특성 중요도를 근거로 왜 그런 결과가 나왔는지 설명해야 한다.

높게 나온 입력 조합 품목=가을무(김장무) / 수거단계=유통/판매 / 재배양식=기타 / 경기도 / 11월 / 10000 m^2 / 100kg → 결과 부적합 (확률 0.5898)

낮게 나온 입력 조합 품목=딸기 / 수거단계=출하 / 재배양식=친환경(인증) 유기 / 인천광역시 / 5월 / 1432 m^2 / 100kg → 결과 적합 (확률 0.9998)

특성 중요도 1위인 수거단계(stage) 기준으로 보면, 유통·판매 단계일수록 오염 노출이 누적돼 부적합 확률이 높아진다. 높은 확률 조합(가을무/유통·판매/11월)은 이 단계에 해당하고, 낮은 확률 조합(딸기/출하/친환경 인증)은 생산 초기이자 친환경 재배라 리스크가 낮은 조건이 겹쳐 있다. 즉 품질 진단에서 본 수거단계별 리스크 차이가 예측 결과에 그대로 반영된 것이다.

11-3 특성 중요도와 대조

```
imp = pd.Series(model.feature_importances_,  
index=X.columns).sort_values(ascending=False)
```

1위 stage (0.231) 2위 month (0.204) 3위 volume (0.172)

1위 stage는 11-2에서 확인한 유통·판매 단계일수록 부적합률이 높다는 패턴과 일치해 이상하지 않다. 2위 month도 계절별 토양·기후 차이로 중금속 축적이 달라진다는 것과 맞아떨어진다. 다만 3위 volume(조사물량)은 품질 진단 단계에서 직접 확인한 항목이 아니라, 물량이 많을수록 표본에 오염된 개체가 섞일 확률이 높아진다는 간접적 해석으로 보완했다.

11-4 모델의 한계

Precision이 0.03에 그친 건 양성이 1.12%뿐인 극심한 불균형 때문에 오탐이 구조적으로 많을 수밖에 없기 때문이다. 또한 재배면적·조사물량의 결측치를 중앙값으로 채운 게 실제 편차를 눌러 세밀한 예측을 방해했을 수 있다. 개선하려면 중금속 성분별 검사 수치나 토양 오염도, 인근 산업시설 여부 같은 환경 데이터를 추가하면 도움이 될 것이다